



S. J. P. N. TRUST'S

HIRASUGAR INSTITUTE OF TECHNOLOGY, NIDASOSHI

Accredited at 'A+' Grade by NAAC

Programmes Accredited by NBA: CSE & ECE

Department of Computer Science & Engineering

MACHINE LEARNING

Module 3:

Chapter 4: Similarity-based Learning

Dr. S. V. Manjaragi

**Asso. Prof. , Dept. of Computer Science & Engg.,
Hirasugar Institute of Technology, Nidasoshi**

Similarity/Instance-based Learning

- *Similarity-based Learning* is a supervised learning technique that **predicts the class label of a test instance by gauging the similarity of this test instance with training instances.**
- Similarity-based learning **refers to a family of instance-based learning which is used to solve both classification and regression problems.**
- Instance-based learning makes prediction by computing distances or similarities between test instance and specific set of training instances local to the test instance in an incremental process.
- Similarity-based classification is useful in various fields such as **image processing, text classification, pattern recognition, bio informatics, data mining, information retrieval, natural language processing, etc.**

Similarity/Instance-based Learning

- Similarity-based classifiers use **similarity measures to locate the nearest neighbors and classify a test instance.**
- This learning mechanism simply **stores all data and uses it only when it needs to classify an unseen instance.**
- The advantage of using this learning is that **processing occurs only when a request to classify a new instance is given.**
- The drawback of this learning is **that it requires a large memory to store the data.**
- Classification of instances is done based on the **measure of similarity in the form of distance functions over data instances.**
- Popular distance metrics used are Hamming distance, Euclidean distance, Manhattan distance, Minkowski distance, Cosine similarity, Mahalanobis distance, Pearson's correlation or correlation similarity, Mean squared difference, Jaccard coefficient, Tanimoto coefficient, etc.

Instance-and Model-based Learning

Table 4.1: Differences between Instance-based Learning and Model-based Learning

Instance-based Learning	Model-based Learning
Lazy Learners	Eager Learners
Processing of training instances is done only during testing phase	Processing of training instances is done during training phase

Instance-based Learning	Model-based Learning
No model is built with the training instances before it receives a test instance	Generalizes a model with the training instances before it receives a test instance
Predicts the class of the test instance directly from the training data	Predicts the class of the test instance from the model built
Slow in testing phase	Fast in testing phase
Learns by making many local approximations	Learns by creating global approximation

- Some examples of Instance-based learning algorithms are:
 1. k-Nearest Neighbor (k-NN)
 2. Variants of Nearest Neighbor learning
 3. Locally Weighted Regression
 4. Learning Vector Quantization (LVQ)
 5. Self-Organizing Map (SOM)
 6. Radial Basis Function (RBF) networks

Nearest-Neighbor Learning

- A natural approach to similarity-based classification is k-Nearest Neighbors (k-NN), which is a **non-parametric method** used for both **classification and regression problems**.
- It predicts the category of the test instance according to the ' k ' training samples which are closer to the test instance and classifies it to that category which has the largest probability.

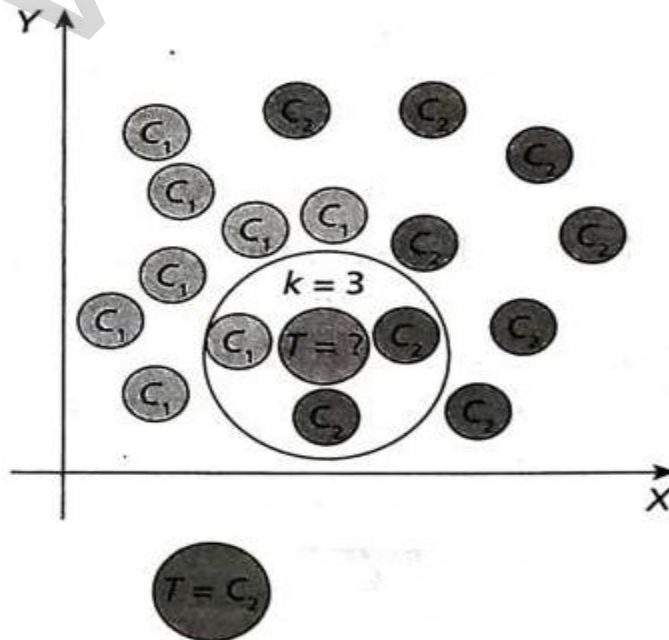


Figure 4.1: Visual Representation of k -Nearest Neighbor Learning

Algorithm 4.1: k-NN

Algorithm 4.1: k-NN

Inputs: Training dataset T , distance metric d , Test instance t , the number of nearest neighbors k

Output: Predicted class or category

Prediction: For test instance t ,

1. For each instance i in T , compute the distance between the test instance t and every other instance i in the training dataset using a distance metric (Euclidean distance).

[Continuous attributes - Euclidean distance between two points in the plane with coordinates (x_1, y_1) and (x_2, y_2) is given as $\text{dist}((x_1, y_1), (x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$]

[Categorical attributes (Binary) - Hamming Distance: If the value of the two instances is same, the distance d will be equal to 0 otherwise $d = 1$.]

2. Sort the distances in an ascending order and select the first k nearest training data instances to the test instance.
3. Predict the class of the test instance by majority voting (if target attribute is discrete valued) or mean (if target attribute is continuous valued) of the k selected nearest instances.

Example 4.1: Consider the student performance training dataset of 8 data instances shown in Table 4.2 which describes the performance of individual students in a course and their CGPA obtained in the previous semesters. The independent attributes are CGPA, Assessment and Project. The target variable is 'Result' which is a discrete valued variable that takes two values 'Pass' or 'Fail'. Based on the performance of a student, classify whether a student will pass or fail in that course.

Table 4.2: Training Dataset

S.No.	CGPA	Assessment	Project Submitted	Result
1.	9.2	85	8	Pass
2.	8	80	7	Pass
3.	8.5	81	8	Pass
4.	6	45	5	Fail
5.	6.5	50	4	Fail
6.	8.2	72	7	Pass
7.	5.8	38	5	Fail
8.	8.9	91	9	Pass

Solution: Given a test instance (6.1, 40, 5) and a set of categories {Pass, Fail} also called as classes, we need to use the training set to classify the test instance using Euclidean distance.

The task of classification is to assign a category or class to an arbitrary instance.

Assign $k = 3$.

Step 1: Calculate the Euclidean distance between the test instance (6.1, 40, and 5) and each of the training instances as shown in Table 4.3.

Table 4.3: Euclidean Distance

S.No.	CGPA	Assessment	Project Submitted	Result	Euclidean Distance
1.	9.2	85	8	Pass	$\sqrt{(9.2 - 6.1)^2 + (85 - 40)^2 + (8 - 5)^2}$ = 45.2063
2.	8	80	7	Pass	$\sqrt{(8 - 6.1)^2 + (80 - 40)^2 + (7 - 5)^2}$ = 40.09501
3.	8.5	81	8	Pass	$\sqrt{(8.5 - 6.1)^2 + (81 - 40)^2 + (8 - 5)^2}$ = 41.17961
4.	6	45	5	Fail	$\sqrt{(6 - 6.1)^2 + (45 - 40)^2 + (5 - 5)^2}$ = 5.001
5.	6.5	50	4	Fail	$\sqrt{(6.5 - 6.1)^2 + (50 - 40)^2 + (4 - 5)^2}$ = 10.05783
6.	8.2	72	7	Pass	$\sqrt{(8.2 - 6.1)^2 + (72 - 40)^2 + (7 - 5)^2}$ = 32.13114
7.	5.8	38	5	Fail	$\sqrt{(5.8 - 6.1)^2 + (38 - 40)^2 + (5 - 5)^2}$ = 2.022375
8.	8.9	91	9	Pass	$\sqrt{(8.9 - 6.1)^2 + (91 - 40)^2 + (9 - 5)^2}$ = 51.23319

Step 2: Sort the distances in the ascending order and select the first 3 nearest training data instances to the test instance. The selected nearest neighbors are shown in Table 4.4.

Table 4.4: Nearest Neighbors

Instance	Euclidean Distance	Class
4	5.001	Fail
5	10.05783	Fail
7	2.022375	Fail

Here, we take the 3 nearest neighbors as instances 4, 5 and 7 with smallest distances.

Step 3: Predict the class of the test instance by majority voting.

The class for the test instance is predicted as 'Fail'.

Weighted k-Nearest-Neighbor Algorithm

- The principle idea of Weighted k-NN is that **k closest neighbors to the test instance are assigned a higher weight in the decision** as compared to neighbors that are farther away from the test instance.
- The idea is that weights are inversely proportional to distances.
- The selected k nearest neighbors can be assigned uniform weights, which means all the instances in each neighborhood are weighted equally or weights can be assigned by the inverse of their distance.

Weighted k-Nearest-Neighbor Algorithm

Algorithm 4.2: Weighted k-NN

Inputs: Training dataset ' T ', Distance metric ' d ', Weighting function $w(i)$, Test instance ' t ', the number of nearest neighbors ' k '

Output: Predicted class or category

Prediction: For test instance t ,

1. For each instance ' i ' in Training dataset T , compute the distance between the test instance t and every other instance ' i ' using a distance metric (Euclidean distance).
[Continuous attributes - Euclidean distance between two points in the plane with coordinates (x_1, y_1) and (x_2, y_2) is given as $\text{dist}((x_1, y_1), (x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$]
[Categorical attributes (Binary) - Hamming Distance: If the values of two instances are the same, the distance d will be equal to 0. Otherwise $d = 1$.]
2. Sort the distances in the ascending order and select the first ' k ' nearest training data instances to the test instance.
3. Predict the class of the test instance by weighted voting technique (Weighting function $w(i)$) for the k selected nearest instances:
 - Compute the inverse of each distance of the ' k ' selected nearest instances.
 - Find the sum of the inverses.
 - Compute the weight by dividing each inverse distance by the sum. (Each weight is a vote for its associated class).
 - Add the weights of the same class.
 - Predict the class by choosing the class with the maximum vote.

Example 4.2: Consider the same training dataset given in Table 4.1. Use Weighted k -NN and determine the class.

Solution:

Step 1: Given a test instance (7.6, 60, 8) and a set of classes {Pass, Fail}, use the training dataset to classify the test instance using Euclidean distance and weighting function.

Assign $k = 3$. The distance calculation is shown in Table 4.5.

Table 4.5: Euclidean Distance

S.No.	CGPA	Assessment	Project Submitted	Result	Euclidean Distance
1.	9.2	85	8	Pass	$\sqrt{(9.2 - 7.6)^2 + (85 - 60)^2 + (8 - 8)^2}$ $= 25.05115$
2.	8	80	7	Pass	$\sqrt{(8 - 7.6)^2 + (80 - 60)^2 + (7 - 8)^2}$ $= 20.02898$
3.	8.5	81	8	Pass	$\sqrt{(8.5 - 7.6)^2 + (81 - 60)^2 + (8 - 8)^2}$ $= 21.01928$

S.No.	CGPA	Assessment	Project Submitted	Result	Euclidean Distance
4.	6	45	5	Fail	$\sqrt{(6 - 7.6)^2 + (45 - 60)^2 + (5 - 8)^2}$ $= 15.38051$
5.	6.5	50	4	Fail	$\sqrt{(6.5 - 7.6)^2 + (50 - 60)^2 + (4 - 8)^2}$ $= 10.82636$
6.	8.2	72	7	Pass	$\sqrt{(8.2 - 7.6)^2 + (72 - 60)^2 + (7 - 8)^2}$ $= 12.05653$
7.	5.8	38	5	Fail	$\sqrt{(5.8 - 7.6)^2 + (38 - 60)^2 + (5 - 8)^2}$ $= 22.27644$
8.	8.9	91	9	Pass	$\sqrt{(8.9 - 7.6)^2 + (91 - 60)^2 + (9 - 8)^2}$ $= 31.04336$

Step 2: Sort the distances in the ascending order and select the first 3 nearest training data instances to the test instance. The selected nearest neighbors are shown in Table 4.6.

Table 4.6: Nearest Neighbors

Instance	Euclidean Distance	Class
4	15.38051	Fail
5	10.82636	Fail
6	12.05653	Pass

Step 3: Predict the class of the test instance by weighted voting technique from the 3 selected nearest instances.

- Compute the inverse of each distance of the 3 selected nearest instances as shown in Table 4.7.

Table 4.7: Inverse Distance

Instance	Euclidean Distance	Inverse Distance	Class
4	15.38051	0.06502	Fail
5	10.82636	0.092370	Fail
6	12.05653	0.08294	Pass

- Find the sum of the inverses.

$$\text{Sum} = 0.06502 + 0.092370 + 0.08294 = 0.24033$$

- Compute the weight by dividing each inverse distance by the sum as shown in Table 4.8.

Table 4.8: Weight Calculation

Instance	Euclidean Distance	Inverse Distance	Weight = Inverse distance/Sum	Class
4	15.38051	0.06502	0.270545	Fail
5	10.82636	0.092370	0.384347	Fail
6	12.05653	0.08294	0.345109	Pass

- Add the weights of the same class.

$$\text{Fail} = 0.270545 + 0.384347 = 0.654892$$

$$\text{Pass} = 0.345109$$

- Predict the class by choosing the class with the maximum vote.

The class is predicted as '**Fail**'.

Nearest Centroid Classifier

Algorithm 4.3: Nearest Centroid Classifier

Inputs: Training dataset T , Distance metric d , Test instance t

Output: Predicted class or category

1. Compute the mean/centroid of each class.
2. Compute the distance between the test instance and mean/centroid of each class (Euclidean Distance).
3. Predict the class by choosing the class with the smaller distance.

Example 4.3: Consider the sample data shown in Table 4.9 with two features x and y . The target classes are 'A' or 'B'. Predict the class using Nearest Centroid Classifier.

Table 4.9: Sample Data

x	y	Class
3	1	A
5	2	A
4	3	A
7	6	B
6	7	B
8	5	B

Solution:

Step 1: Compute the mean/centroid of each class. In this example there are two classes called 'A' and 'B'.

Centroid of class 'A' = $(3 + 5 + 4, 1 + 2 + 3)/3 = (12, 6)/3 = (4, 2)$

Centroid of class 'B' = $(7 + 6 + 8, 6 + 7 + 5)/3 = (21, 18)/3 = (7, 6)$

Now given a test instance (6, 5), we can predict the class.

Step 2: Calculate the Euclidean distance between test instance (6, 5) and each of the centroid.

$$\text{Euc_Dist}[(6, 5) ; (4, 2)] = \sqrt{(6-4)^2 + (5-2)^2} = \sqrt{13} = 3.6$$

$$\text{Euc_Dist}[(6, 5) ; (7, 6)] = \sqrt{(6-7)^2 + (5-6)^2} = \sqrt{2} = 1.414$$

The test instance has smaller distance to class B. Hence, the class of this test instance is predicted as 'B'.

Locally Weighted Regression (LWR)

- Locally Weighted Regression (LWR) is a non-parametric supervised learning algorithm that **performs local regression by combining regression model with nearest neighbor's model.**
- LWR is also referred to as a memory-based method as it requires training data while prediction but **uses only the training data instances locally around the point of interest.**
- **Using nearest neighbors algorithm, we find the instances that are closest to a test instance and fit linear function to each of those 'k' nearest instances in the local regression model.**
- The key idea is that we need to approximate the linear functions of all 'k' neighbors that minimize the error such that the prediction line is no more linear but rather it is a curve.

Locally Weighted Regression (LWR)

- Ordinary linear regression finds out a linear relationship between the input x and the output y , Given training dataset T ,
- Hypothesis function $h_{\beta}(x)$, the predicted target output is a linear function where β_0 is the intercept and β_1 is the coefficient of x .
- It is given in Eq. (4.1) as,

$$h_{\beta}(x) = \beta_0 + \beta_1 x \quad (4.1)$$

- The cost function is such that it minimizes the error difference between the predicted value $h_{\beta}(x)$, and true value 'y' and it is given as in Eq. (4.2).

$$J(\beta) = \frac{1}{2} \sum_{i=1}^m (h_{\beta}(x_i) - y_i)^2 \quad (4.2)$$

- where ' m ' is the number of instances in the training dataset.

Locally Weighted Regression (LWR)

- Now the cost function is modified for locally weighted linear regression including the weights only for the nearest neighbor points. Hence, the cost function is given as in Eq. (4.3).

$$J(\beta) = \frac{1}{2} \sum_{i=1}^m w_i (h_{\beta}(x_i) - y_i)^2 \quad (4.3)$$

- where w_i is the weight associated with each x_i .
- The **weight function used is a Gaussian kernel that gives a higher value for instances that are close to the test instance**, and for instances far away, it tends to zero but never equals to zero.

w_i is computed in Eq. (4.4) as,

$$w_i = e^{\frac{-(x_i - x)^2}{2\tau^2}} \quad (4.4)$$

where, τ is called the bandwidth parameter and controls the rate at which w_i reduces to zero with distance from x_i .

Example 4.4: Consider a simple example with four instances shown in Table 4.10 and apply locally weighted regression.

Table 4.10: Sample Table

S.No.	Salary (in lakhs)	Expenditure (in thousands)
1.	5	25
2.	1	5
3.	2	7
4.	1	8

Solution: Using linear regression model assuming we have computed the parameters:

$$\beta_0 = 4.72, \beta_1 = 0.62$$

Given a test instance with $x = 2$, the predicted y' is:

$$y' = \beta_0 + \beta_1 x = 4.72 + 0.62 \times 2 = 5.96$$

Applying the nearest neighbor model, we choose $k = 3$ closest instances.

Table 4.11 shows the Euclidean distance calculation for the training instances.

Table 4.11: Euclidean Distance Calculation

S.No.	x = Salary (in lakhs)	y = Expenditure (in thousands)	Euclidean Distance
1.	5	25	$\sqrt{(5-2)^2} = 3$
2.	1	5	$\sqrt{(1-2)^2} = 1$
3.	2	7	$\sqrt{(2-2)^2} = 0$
4.	1	8	$\sqrt{(1-2)^2} = 1$

Instances 2, 3 and 4 are closer with smaller distances.

The mean value = $(5 + 7 + 8)/3 = 20/3 = 6.67$.

Using Eq. (4.4) compute the weights for the closest instances, using the Gaussian kernel,

$$w_i = e^{\frac{-(x_i - x)^2}{2\tau^2}}$$

Hence the weights of the closest instances is computed as follows;

Weight of Instance 2 is:

$$w_2 = e^{\frac{-(x_2 - x)^2}{2\tau^2}} = e^{\frac{-(1-2)^2}{2 \times 0.4^2}} = e^{-3.125} = 0.043$$

Weight of Instance 3 is:

$$w_3 = e^{\frac{-(x_3-x)^2}{2r^2}} = e^{\frac{-(2-2)^2}{2 \times 0.4^2}} = e^0 = 1 \text{ [} w_3 \text{ is closer hence gets a higher weight value]}$$

Weight of Instance 4 is:

$$w_4 = e^{\frac{-(x_4-x)^2}{2r^2}} = e^{\frac{-(1-2)^2}{2 \times 0.4^2}} = e^{-3.125} = 0.043$$

The predicted output for the three closer instances is given as follows:

The predicted output of Instance 2 is:

$$y_2' = h_{\beta}(x_2) = \beta_0 + \beta_1 x_2 = 4.72 + 0.62 \times 1 = 5.34$$

The predicted output of Instance 3 is:

$$y_3' = h_{\beta}(x_3) = \beta_0 + \beta_1 x_2 = 4.72 + 0.62 \times 2 = 5.96$$

The predicted output of Instance 4 is:

$$y_4' = h_{\beta}(x_4) = \beta_0 + \beta_1 x_2 = 4.72 + 0.62 \times 1 = 5.34$$

The error value is calculated as:

$$J(\beta) = \frac{1}{2} \sum_{i=1}^m w_i (h_{\beta}(x_i) - y_i)^2 = \frac{1}{2} (0.043(5.34 - 5)^2 + 1(5.96 - 7)^2 + 0.043(5.34 - 8)^2) = 0.6953$$

Now, we need to adjust this cost function to minimize the error difference and get optimal β parameters.